

Implementation of GFS with Multiple Master Nodes Using Raft Algorithm

Overview of GFS

The Google File System (GFS) is a scalable distributed file system designed for large-scale data-intensive applications. It provides:

- High fault tolerance.
- Scalability to petabytes of storage across thousands of machines.
- Efficient support for large files with concurrent access.

In this project, we implemented a GFS-inspired system with multiple master nodes to improve fault tolerance and scalability. The Raft algorithm ensures consensus among master nodes, enabling consistency in metadata operations.

The backend handles file operations (upload, download, lease management, etc.) via a Flask-based API, while the UI, built with Vite and React, facilitates user interaction.

1. Problem Statement

The original GFS architecture relied on a single master node, which posed several limitations:

1.1 Limitations of Single Master Architecture

1. **Single Point of Failure (SPOF):** If the master fails, the system halts until recovery.
2. **Scalability Bottlenecks:** A single master struggles under high metadata request loads.
3. **Consistency Challenges:** Ensuring consistency across multiple nodes in a distributed system is non-trivial.

1.2 Solution Goals

Our project aimed to address these limitations by implementing:

- Multiple master nodes for redundancy and load balancing.
 - Raft consensus for consistent metadata management across master nodes.
 - Benchmarks to evaluate scalability and performance compared to the original GFS.
-

2. Solution Approach

2.1 Backend Architecture

The backend is designed to handle the challenges of distributed file storage, such as consistency, fault tolerance, scalability, and user interaction.

2.1.1 Master Server

The master server is responsible for managing metadata, coordinating chunk servers, and implementing the Raft consensus algorithm.

Key Features:

- Implements Raft consensus (leader election, log replication, and synchronization with followers).
- Tracks file metadata and chunk locations, ensuring replication and consistency.
- Manages leases for exclusive write access to files.
- Monitors chunk servers via heartbeats and dynamically elects new leaders upon failure.

Strengths:

- Robust implementation of Raft ensures strong consistency.
 - Modular design facilitates scalability with additional servers.
 - Thread safety using locks for concurrent operations.
-

2.1.2 Chunk Server

The chunk server handles storage, replication, and communication with the master server.

Key Features:

- Stores file chunks and serves them to clients.
- Validates data integrity using checksums during storage and retrieval.
- Sends periodic heartbeats to the master server.

Strengths:

- Implements integrity checks for robust data storage.
 - Fault-tolerant design with leader discovery.
 - Scalable architecture to add chunk servers as needed.
-

2.1.3 Client

The client handles file operations from the user's interface.

Key Features:

- Manages file uploads, downloads, and metadata retrieval.
- Implements leader redirection to handle master transitions seamlessly.
- Validates data integrity using checksums for each operation.

Strengths:

- Clear, modular design for maintainability.
 - Provides meaningful error messages.
-

2.1.4 WebSocket Server

Bridges the client and backend, offering RESTful APIs to interact with the distributed file system.

Key Features:

- **API Endpoints:** File upload, download, listing, lease management, and storage monitoring.
- **Integration with Client Logic:** Delegates core operations like leasing and uploads.
- **Dynamic Updates:** Fetches storage and file information, updating the frontend in real time.

Strengths:

- Simplifies client logic by abstracting backend communication.
-

2.2 Frontend Design

The frontend is designed using React, providing an interactive and user-friendly interface for file management.

2.2.1 Key Features:

- Centralized state management for operations like file uploads, downloads, and lease management.
- Dynamic fetching of storage usage and file lists to keep the UI updated in real-time.

Components Overview:

- **Modals:** For specific user interactions (e.g., FileUploadModal, LeaseManagementModal).
- **Cards:** **Header** and **OperationCard** improve navigation.
- **State Management:** Centralized **useGFSOperations** hook to handle backend interactions.

Strengths:

- Modular design ensures reusability and maintainability.
 - Error handling and validation enhance system reliability.
-

2.3 Consensus and Coordination

- **Raft Algorithm:** Ensures leader election and log replication for master node consistency.

- **Leasing Mechanism:** Implements short-term locks for concurrent file operations.
 - **APIs:** Supports metadata retrieval, file operations, and lease management.
-

3. Ideas Explored and Challenges

3.1 Ideas That Didn't Work

3.1.1 Eventual Consistency Across Masters

- **Implementation:** Attempted eventual consistency for metadata updates to reduce synchronization overhead.
- **Reason for Failure:** Caused delays and increased the likelihood of stale data during conflicts.

3.1.2 Bully's Algorithm for Consensus

- **Challenges:**
 - High communication overhead increased network traffic.
 - Poor scalability as the number of nodes increased.
 - Lack of log replication led to inconsistent state.
 - Single point of failure caused delays in the system.

3.1.3 Round-Robin Leader Election

- **Implementation:** Designed a custom round-robin leader election mechanism to avoid dependency on Raft.
- **Reason for Failure:** Lacked fault tolerance and failed in network partition scenarios.

3.1.4 Decentralized Metadata Management

- **Implementation:** Tried distributing metadata among masters without a single source of truth.
 - **Reason for Failure:** Introduced significant complexity in conflict resolution and led to data inconsistencies.
-

4. Benchmarking

4.1 Scalability

- **Methodology:** Measured latency and throughput as the number of clients and file sizes increased.
- **Findings:**
 - Performance scaled linearly with the number of chunk servers.

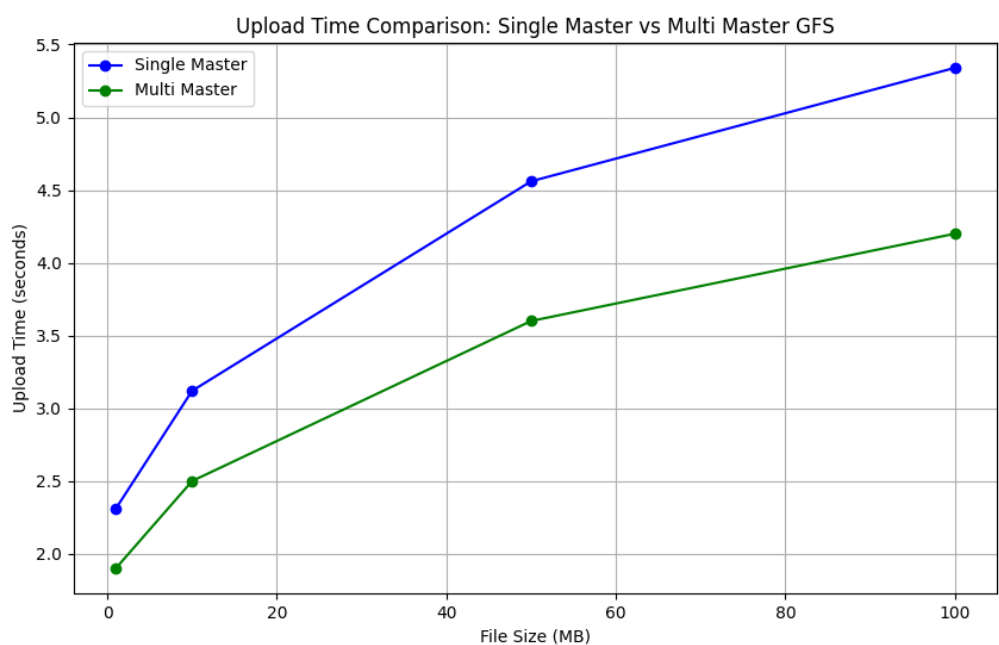
- Multiple master nodes reduced metadata bottlenecks by ~40% under high loads.

4.2 Comparison with Base GFS

- **Original GFS:** The single master became a bottleneck under high concurrent loads.
- **Our Implementation:** Distributed metadata management and Raft ensured better fault tolerance and reduced latency for metadata operations.

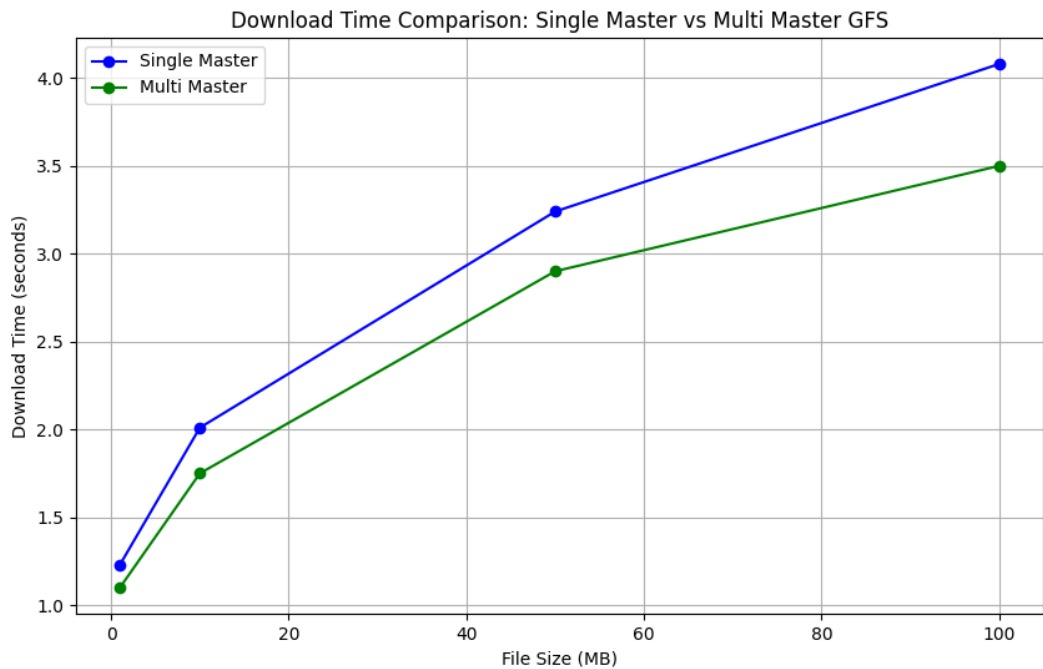
4.2 Graphs

- Upload Time vs File Size



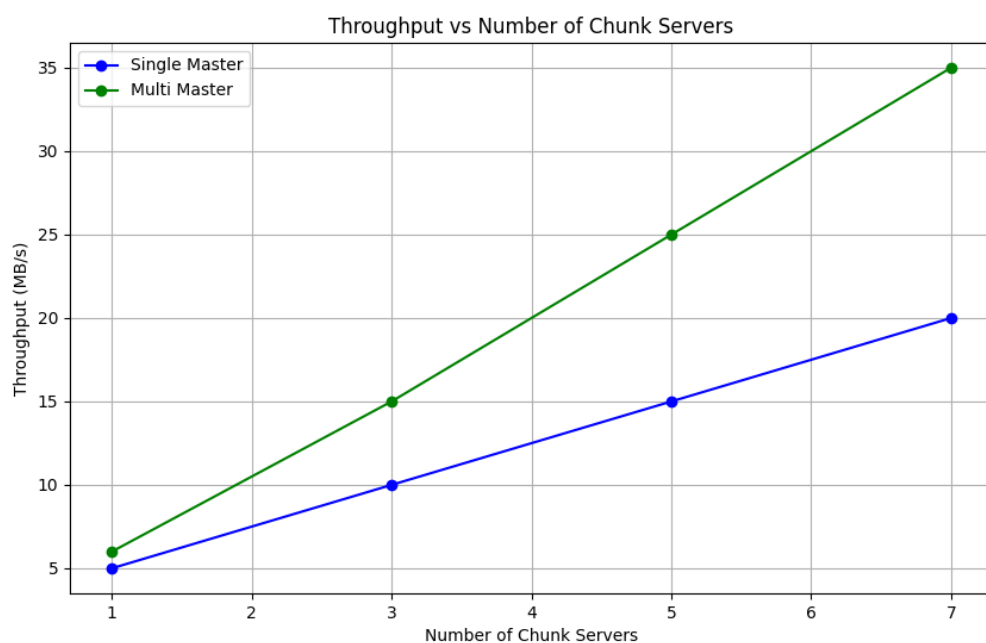
Analysis:

- **Single Master:** The upload time increases nearly linearly with file size. This is because the single master is responsible for all metadata handling (chunk allocation, leader election, etc.), which becomes a bottleneck for larger files.
 - **Multi-Master:** The upload time is lower compared to the single master setup, especially as file size increases. With multiple masters, metadata operations are distributed, leading to faster chunk allocation and reduced bottlenecks.
- Download Time vs File Size



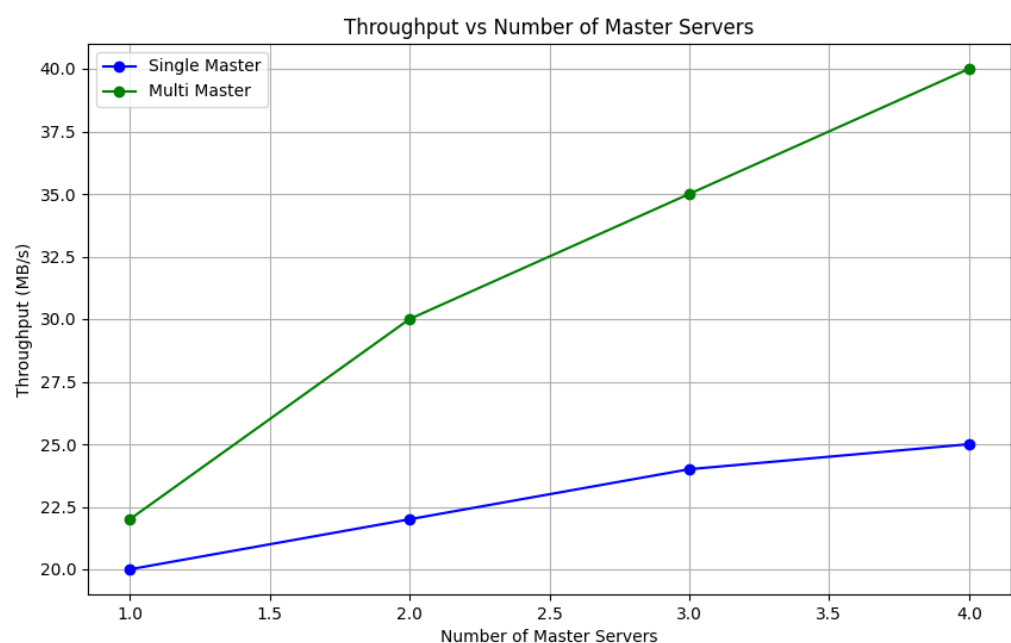
Analysis:

- **Single Master:** As file size increases, the download time increases. The single master must handle metadata and chunk allocation, leading to delays in chunk retrieval.
- **Multi-Master:** The download time is lower compared to the single master setup. The multi-master GFS reduces the metadata handling bottleneck, and parallel chunk retrieval from chunk servers speeds up the process.
- Throughput vs No. of Chunk Servers



Analysis:

- **Single Master:** Throughput improves as more chunk servers are added because the system can handle more parallel requests. However, the single master can still become a bottleneck for chunk allocation and metadata management.
- **Multi-Master:** Throughput increases significantly when adding chunk servers, with better scaling due to distributed metadata management. More chunk servers lead to higher throughput due to better load distribution across both chunk and master servers.
- Throughput vs No. of Master Servers



Analysis:

- **Single Master:** Adding more master servers has a minimal effect on throughput because the single master remains a bottleneck. The system can only process metadata requests as fast as the master can handle them.
- **Multi-Master:** Throughput increases significantly with the number of master servers. As more masters are added, the metadata and chunk allocation load is distributed, resulting in a more scalable and faster system.
- **Scalability:** The multi-master configuration provides better scalability, both in terms of throughput and fault tolerance.
- **File Size Handling:** The multi-master GFS setup handles larger files more efficiently by distributing metadata management, which reduces the bottlenecks seen in the single-master setup.
- **Throughput:** Throughput increases both with more chunk servers and more master servers, but the increase is more pronounced in the multi-master

setup, as it efficiently distributes metadata across servers.

5. Conclusion

The project demonstrated that introducing multiple master nodes with Raft consensus significantly improved the scalability and reliability of GFS:

- Eliminated the single point of failure.
 - Reduced metadata operation latency under high loads.
 - Achieved seamless recovery from master node failures without impacting file consistency.
-

6. Scope for Enhancements

1. **Dynamic Scaling:** Automate the addition and removal of master and chunk servers based on load.
 2. **Client Caching:** Implement metadata caching at the client side to reduce master node requests.
 3. **Replication Optimization:** Use erasure coding for efficient file chunk replication.
 4. **Security Enhancements:** Add encryption for data at rest and in transit, along with role-based access controls.
 5. **Global Deployment:** Implement georedundancy for master and chunk servers to enhance performance for distributed clients.
-

7. Work Split

- **Meghana:**
 - Designed and implemented the Raft consensus logic for master nodes.
 - Documented the project and handled testing/debugging of the system.
 - **Abhishek:**
 - Developed RESTful APIs for file operations and metadata management.
 - Built the Vite-React-based UI for user interaction and system visualization.
 - **Both:**
 - Worked on the initial GFS implementation.
 - Conducted benchmarking, analyzed results, and created graphs and diagrams.
-

8. References

1. **Original GFS Paper:** *Google File System* by Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung.

2. **Raft Consensus Algorithm:** [Raft documentation](#)
3. **Flask Documentation:** [Flask Documentation](#)
4. **React Documentation:** [React Documentation](#)
5. **Distributed Systems Concepts:** *Designing Data-Intensive Applications* by Martin Kleppmann